RAMAIAH
Institute of Technology

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSL75: SKILL ENHANCEMENT LAB - GENERATIVE AI APPLICATION LAB
TERM: August-December 2026

SKILL Lab - Tutorial

Submitted to
Dr. S. Rajarajeswari
Associate Professor

TEAM MEMBERS

Sl. No	USN	Name	Marks
1.	1MS23CS012	Aditya Karan	
2.	1MS23CS016	Aggarwal Shivansh	
3.	1MS23CS019	Amay Ranjan	
4.	1MS23CS024	Anjan Padaki	

M.S. RAMAIAH INSTITUTE OF TECHNOLOGY
(Autonomous Institute, Affiliated to VTU)

CONTENTS

CSL75 - Skill Enhancement Laboratory: Generative AI

Experiment / Program	Page
Cover Page	1
Contents	2
1. MULTIMODAL GENERATIVE AI - TEXT, IMAGE AND SOUND	3
2. USING WORD EMBEDDINGS TO ENRICH GENERATIVE AI PROMPTS	4
3. STUDENT PASS/FAIL PREDICTION USING A TENSORFLOW DNN	5
4. SUMMARISING AN ACADEMIC REPORT USING HUGGING FACE	6
5. PRETRAINED WORD VECTORS, VECTOR ARITHMETIC AND PCA VISUALISATION	7
6. TRAINING A CUSTOM WORD2VEC MODEL ON A MEDICAL CORPUS	8
7. LANGCHAIN PROMPT TEMPLATE WITH A GOOGLE DRIVE TEXT DOCUMENT	9
8. SENTIMENT ANALYSIS USING A HUGGING FACE PIPELINE	10
9. STRUCTURED INSTITUTION INFORMATION EXTRACTION USING LANGCHAIN AND PYDANTIC	11
10. RETRIEVAL-BASED IPC QUESTION-ANSWERING CHATBOT	13
11. TIME-SERIES FORECASTING USING A PYTORCH RNN	15
12. CHARACTER-LEVEL TEXT GENERATION USING AN LSTM	17

PROGRAM 1: MULTIMODAL GENERATIVE AI - TEXT, IMAGE AND SOUND

Aim: To use generative AI tools to create a short story or poem, an image, and a 15-30 second sound or music clip around one theme, and to analyse how consistently the three outputs represent that theme.

Theory: Multimodal generative AI systems create different kinds of content from instructions called prompts. Text-generation models produce language, text-to-image systems create visual scenes, and audio-generation systems produce sound or music. Using one shared theme and consistent descriptive details helps align the three outputs. The generated media should still be reviewed for factual accuracy, stereotypes, copyright concerns, and unintended content.

Algorithm / Procedure

- Choose a common theme; for this demonstration, use “A cleaner and greener future”.
- Ask a text-generation tool for a short poem or story that mentions clean energy, trees, and a hopeful future.
- Use a text-to-image tool with a prompt based on the same setting, objects, colour mood, and message.
- Use an AI audio or music tool to generate a 15-30 second instrumental clip matching the mood.
- Compare the three outputs for shared theme, key objects, emotional tone, and any contradictory details.
- Record the tool names, prompts, outputs, and any edits. Use only content permitted by the tool and the course rules.

Tools and Libraries: Generative AI text tool, text-to-image tool, and AI audio/music generation tool. No Python package is required by the original exercise; the work is tool-based.

Program / Practical Procedure

This exercise is tool-based in the course specification; the working procedure, prompts, evaluation, and reportable observations are provided above and below. No Python script is required by the stated exercise.

Expected Output / Observations

Illustrative text output (original sample): “Let sunlight wake the rooftops bright, / Let rivers run in clear daylight; / Plant a tree and let it grow, / Give tomorrow room to glow.” The image prompt should depict a clean-energy neighbourhood with trees and sunlight. The audio prompt should request a gentle, optimistic 15-30 second instrumental. Replace these examples with the outputs actually produced in your chosen tools before final submission.

Sample Prompts for Reproduction

Text prompt: Write a four-line optimistic poem about a cleaner future, mentioning sunlight, trees, clean energy, and healthier cities.

Image prompt: A hopeful sustainable city at sunrise, rooftop solar panels, wind turbines in the distance, trees along clean streets, warm golden light, cohesive illustrated editorial style, no written text.

Audio prompt: Create a 15-30 second gentle, optimistic instrumental soundscape with warm piano, soft strings, and light nature ambience; no vocals; mood should suggest renewal and hope.

Multimodal Alignment Checklist

Dimension	What to compare
Theme	Do all three artefacts communicate a cleaner, greener future?
Objects / details	Do the image and text share motifs such as sunlight, trees, or clean energy?
Mood	Does the sound match the optimistic tone of the poem and image?
Ethics / quality	Check for unwanted content, misleading details, bias, and rights restrictions.

Result / Conclusion

The exercise demonstrates how a shared theme can be represented through text, visual composition, and sound. Alignment is strongest when all outputs preserve the same setting and emotional tone. The evaluation is qualitative and should be based on the actual generated artefacts.

PROGRAM 2: USING WORD EMBEDDINGS TO ENRICH GENERATIVE AI PROMPTS

Aim: To retrieve semantically related words using pretrained word embeddings, use those words to enrich a prompt, and compare the responses generated from the original and enriched prompts.

Theory: A word embedding represents a word as a numeric vector learned from language data. Words used in related contexts often have vectors that are close under cosine similarity. Related words can help expand a prompt with useful concepts, examples, and terminology. Similarity is based on the training corpus and does not guarantee that every retrieved word is appropriate for a given task.

Algorithm / Procedure

- Load a pretrained embedding model and select a target concept.
- Retrieve the five nearest words using cosine similarity.
- Construct an original prompt and an enriched prompt containing the retrieved concepts.
- Submit both prompts to the same generative AI model with the same settings.
- Compare detail, relevance, clarity, repetition, and any irrelevant concepts introduced by enrichment.

Tools and Libraries: Python 3.10+, gensim, and optionally the OpenAI Python package if automated response generation is used. The embedding model is downloaded on first use.

Program / Practical Procedure

```
import os
import gensim.downloader as api
# One-time download may be required on first run.
print("Loading pretrained word embeddings...")
model = api.load("glove-wiki-gigaword-50")
word = "technology"
related = model.most_similar(word, topn=5)
terms = [item[0] for item in related]
original = "Explain technology in simple terms for a first-year student."
enriched = (
    "Explain technology in simple terms for a first-year student. "
    f"Also relate it to {', '.join(terms)} and give practical examples."
)
print("Target word:", word)
print("Related words:", related)
print("\nORIGINAL PROMPT:\n", original)
print("\nENRICHED PROMPT:\n", enriched)
# Optional: generate both responses with the same model/API key.
if os.getenv("OPENAI_API_KEY"):
    from openai import OpenAI
    client = OpenAI()
    for label, prompt in [("Original", original), ("Enriched", enriched)]:
        response = client.responses.create(
            model=os.getenv("OPENAI_MODEL", "gpt-4.1-mini"), input=prompt
        )
        print(f"\n{label} response:\n{response.output_text}")
else:
    print("Set OPENAI_API_KEY to generate both responses automatically, or paste the two prompts into the same AI tool.")
```

Expected Output / Observations

The program prints five semantically related words and both prompt versions. If an API key is configured, it also prints the model responses. Actual related words and response text depend on the downloaded embedding model and the chosen language model.

Result / Conclusion

The enriched prompt gives the model more related concepts to cover. Compare both responses using the same model so that the prompt is the main variable. Do not assume that a longer response is automatically more relevant.

PROGRAM 3: STUDENT PASS/FAIL PREDICTION USING A TENSORFLOW DNN

Aim: To build a TensorFlow deep neural network that predicts whether a student passes or fails using study hours and attendance percentage.

Theory: A deep neural network consists of layers of trainable neurons. Here, study hours and attendance are numerical input features, and a sigmoid output estimates the probability of passing. Binary cross-entropy is used as the loss function, while a held-out test set gives a basic estimate of generalisation. For demonstration, the sample data are synthetic; they are not real student records.

Algorithm / Procedure

- Generate or load records containing weekly study hours, attendance percentage, and pass/fail labels.
- Scale input features to comparable ranges and split the data into training and testing sets.
- Create a Keras sequential model with dense layers and a sigmoid output unit.
- Compile with the Adam optimiser and binary cross-entropy loss.
- Train the model, evaluate test accuracy, and predict the outcome for a sample student.

Tools and Libraries: Python 3.10+, TensorFlow, NumPy, and scikit-learn.

Program / Practical Procedure

```
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
np.random.seed(42)
tf.random.set_seed(42)
# Synthetic practice data: [study hours per week, attendance percentage]
hours = np.random.uniform(0, 30, 600)
attendance = np.random.uniform(40, 100, 600)
score = 0.65 * (hours / 30) + 0.35 * (attendance / 100)
labels = (score + np.random.normal(0, 0.08, 600) >= 0.58).astype(int)
X = np.column_stack([hours, attendance]).astype("float32")
y = labels.astype("float32")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X_train, y_train, epochs=35, batch_size=32, verbose=0)
prob = model.predict(X_test, verbose=0).ravel()
pred = (prob >= 0.5).astype(int)
print("Test accuracy:", round(accuracy_score(y_test, pred), 3))
new_student = scaler.transform([[12, 82]])
p = float(model.predict(new_student, verbose=0)[0, 0])
print(f"Pass probability for 12 study hours and 82% attendance: {p:.3f}")
print("Prediction:", "PASS" if p >= 0.5 else "FAIL")
```

Expected Output / Observations

The program prints a test-set accuracy, a predicted pass probability, and a PASS/FAIL label for the sample input (12 weekly study hours and 82% attendance). Exact metric values can vary slightly with library versions and execution settings.

Result / Conclusion

The DNN completes a binary classification workflow from feature preparation to evaluation. Since the demonstration data are synthetic and use an assumed label rule, its predictions must not be interpreted as an assessment of actual students.

PROGRAM 4: SUMMARISING AN ACADEMIC REPORT USING HUGGING FACE

Aim: To use a pretrained Hugging Face summarisation model to convert a long academic report into a concise summary for faster review.

Theory: Text summarisation condenses a document while attempting to preserve its most important information. Abstractive summarisation models can generate new phrasing rather than merely selecting complete sentences. Input size is constrained by model limits, so long documents should be split into sections and summarised in stages. The resulting summary should be checked against the original report.

Algorithm / Procedure

- Read an academic report from a UTF-8 text file, or use a short demonstration passage.
- Load a pretrained summarisation pipeline.
- Split the document into manageable word chunks if it is long.
- Summarise each chunk and print the concise output.
- Review the summary for missing conclusions, incorrect numbers, and unsupported statements.

Tools and Libraries: Python 3.10+, transformers, and PyTorch. Internet access is needed the first time model weights are downloaded.

Program / Practical Procedure

```
from pathlib import Path
from transformers import pipeline
# Usage: python summarise_report.py report.txt
path = Path("academic_report.txt")
if len(__import__("sys").argv) > 1:
    path = Path(__import__("sys").argv[1])
if path.exists():
    text = path.read_text(encoding="utf-8")
else:
    text = ("This study investigates the effect of regular revision and class "
            "attendance on student learning. The methods include a survey, "
            "data cleaning, and statistical analysis. Results suggest that "
            "consistent practice is associated with better performance. "
            "The report recommends structured revision and timely feedback.")
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
# Small word chunks help keep each request within model input limits.
words = text.split()
chunks = [" ".join(words[i:i + 450]) for i in range(0, len(words), 450)]
summaries = []
for chunk in chunks:
    if len(chunk.split()) < 35:
        summaries.append(chunk)
        continue
    result = summarizer(chunk, max_length=110, min_length=25, do_sample=False)
    summaries.append(result[0]["summary_text"])
print("\nACADEMIC REPORT SUMMARY\n")
print(" ".join(summaries))
```

Expected Output / Observations

The pipeline prints a concise summary. On the first run, the model and supporting files may be downloaded. The demonstration passage is used only if no input file exists at the specified path.

Result / Conclusion

The experiment demonstrates pretrained summarisation with minimal model code. Summary quality depends on input quality and chunking; any academic or numerical claims should be checked against the full report.

PROGRAM 5: PRETRAINED WORD VECTORS, VECTOR ARITHMETIC AND PCA VISUALISATION

Aim: To explore pretrained word vectors through vector arithmetic, visualise ten technology-related words using PCA, and print five semantically similar words for a given input.

Theory: Pretrained word vectors place words in a continuous numerical space. Cosine similarity measures closeness between vectors, while vector arithmetic can reveal approximate relationships learned from the corpus. PCA projects high-dimensional vectors onto two dimensions for visual inspection. A two-dimensional view is an approximation and may not preserve every original relationship.

Algorithm / Procedure

- Load a pretrained GloVe word-vector model.
- Perform a vector-arithmetic query and inspect the nearest words.
- Select ten technology-related terms and retrieve their vectors.
- Apply PCA to reduce the vectors to two dimensions and plot labelled points.
- Retrieve the five words most similar to the user-selected input word.

Tools and Libraries: Python 3.10+, gensim, scikit-learn, and matplotlib.

Program / Practical Procedure

```
import gensim.downloader as api
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
model = api.load("glove-wiki-gigaword-50")
print("Vector arithmetic (king + woman - man):")
print(model.most_similar(positive=["king", "woman"], negative=["man"], topn=5))
words = ["computer", "software", "internet", "network", "data",
        "algorithm", "program", "hardware", "digital", "information"]
valid = [w for w in words if w in model]
vectors = [model[w] for w in valid]
points = PCA(n_components=2, random_state=42).fit_transform(vectors)
plt.figure(figsize=(8, 6))
plt.scatter(points[:, 0], points[:, 1])
for word, (x, y) in zip(valid, points):
    plt.annotate(word, (x, y), xytext=(4, 4), textcoords="offset points")
plt.title("PCA of Pretrained Technology Word Vectors")
plt.xlabel("Principal component 1")
plt.ylabel("Principal component 2")
plt.tight_layout()
plt.savefig("word_embeddings_pca.png", dpi=160)
plt.close()
query = input("Enter a word for similarity search (default: computer): ").strip() or "computer"
if query not in model:
    print("The word is not present in this model vocabulary.")
else:
    print("Five similar words:")
    for word, score in model.most_similar(query, topn=5):
        print(f"{word}: {score:.3f}")
print("Plot saved to word_embeddings_pca.png")
```

Expected Output / Observations

The program prints the top words from the vector arithmetic query, the five words most similar to the selected input, and saves `word_embeddings_pca.png`. Nearest-word order depends on the embedding model.

Result / Conclusion

The experiment demonstrates vector relationships and dimensionality reduction. Clusters can suggest semantic relationships, but results should be interpreted carefully because pretrained vectors reflect the vocabulary and biases of their training corpus.

PROGRAM 6: TRAINING A CUSTOM WORD2VEC MODEL ON A MEDICAL CORPUS

Aim: To train a Word2Vec model on a small medical/healthcare corpus and inspect how the learned embeddings represent domain-specific terms.

Theory: Word2Vec learns vector representations from the context in which words occur. The Skip-Gram architecture predicts context words from a target word. With a domain-specific corpus, medical terms that occur in related contexts may become close in vector space. A small corpus demonstrates the process but is too limited for reliable clinical or research conclusions.

Algorithm / Procedure

- Create a short collection of domain-specific sentences.
- Convert sentences to lowercase tokens and remove simple stop words.
- Train a Skip-Gram Word2Vec model with a small vector size and a chosen context window.
- Query nearest neighbours for selected words and calculate similarities between pairs.
- Interpret the results as corpus-specific patterns, not clinical knowledge.

Tools and Libraries: Python 3.10+ and gensim.

Program / Practical Procedure

```
import re
from gensim.models import Word2Vec
corpus = [
 "The doctor examines the patient and diagnoses infection.",
 "The physician treats the patient with medication.",
 "The nurse monitors fever and other symptoms.",
 "Antibiotic medication can treat bacterial infection.",
 "The hospital doctor reviews clinical records.",
 "The patient reports cough, fever, and headache.",
 "The surgeon performs surgery in the hospital.",
 "Laboratory tests support diagnosis and treatment.",
 "The nurse records vital signs and patient symptoms.",
 "Medication and treatment depend on the diagnosis."
]
stop = {"the", "and", "with", "can", "on", "in", "of", "to"}
tokens = []
for sentence in corpus:
    words = re.findall(r"[a-z]+", sentence.lower())
    tokens.append([w for w in words if w not in stop])
model = Word2Vec(sentences=tokens, vector_size=40, window=4,
                 min_count=1, sg=1, workers=1, seed=42, epochs=100)
for target in ["doctor", "patient", "medication", "fever"]:
    print(f"\nSimilar to {target}:")
    print(model.wv.most_similar(target, topn=3))
print("Similarity (doctor, physician):",
      round(model.wv.similarity("doctor", "physician"), 3))
print("Vocabulary size:", len(model.wv))
```

Expected Output / Observations

The program prints nearest-neighbour words for doctor, patient, medication, and fever, a cosine-similarity score for doctor/physician, and the vocabulary size. The exact scores are learned from the small corpus and random initialisation.

Result / Conclusion

The custom model shows how co-occurrence patterns can encode domain associations. Because the training corpus is small, individual similarity results may be unstable and should not be used for medical decision-making.

PROGRAM 7: LANGCHAIN PROMPT TEMPLATE WITH A GOOGLE DRIVE TEXT DOCUMENT

Aim: To load a publicly shared text document from Google Drive, use a Cohere language model through LangChain, and format the response with a reusable prompt template.

Theory: LangChain provides reusable prompt templates and composable chains that connect prompt formatting, model invocation, and output parsing. A document can be provided as context so the model answers in a requested structure. The input file should be accessible to the script and the API key should be stored in an environment variable rather than hard-coded.

Algorithm / Procedure

- Upload a UTF-8 `.txt` file to Google Drive and enable link access for the account running the script.
- Copy the file ID from the share link and store it in `GOOGLE_DRIVE_FILE_ID`.
- Store the Cohere API key in `COHERE_API_KEY`.
- Download the text, construct a prompt template, and connect it to ChatCohere and an output parser.
- Run the chain and review the response against the source document.

Tools and Libraries: Python 3.10+, requests, langchain-core, langchain-cohere, and langchain. A Cohere API key and a publicly accessible Google Drive text file are required.

Program / Practical Procedure

```
import os
import requests
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_cohere import ChatCohere
file_id = os.environ["GOOGLE_DRIVE_FILE_ID"]
url = f"https://drive.google.com/uc?export=download&id={file_id}"
response = requests.get(url, timeout=30)
response.raise_for_status()
document = response.text.strip()
if not document or "<html" in document[:500].lower():
    raise ValueError("Google Drive did not return plain text; check sharing and file ID.")
prompt = PromptTemplate.from_template(
    "You are an academic assistant. Use only the document below.\n"
    "Return exactly these headings: Summary, Key Points, Conclusion.\n\n"
    "DOCUMENT:\n{document}\n\nUser request: {question}"
)
llm = ChatCohere(model="command-r", temperature=0)
chain = prompt | llm | StrOutputParser()
question = input("What should be extracted from the document? ").strip()
answer = chain.invoke({"document": document[:12000], "question": question})
print("\nFORMATTED RESPONSE\n")
print(answer)
```

Expected Output / Observations

The program downloads the publicly shared text, asks the user what should be extracted, and prints a response with Summary, Key Points, and Conclusion headings. A Google Drive confirmation/login page indicates that the file permissions or download URL need attention.

Result / Conclusion

The experiment demonstrates document-grounded prompting and a LangChain model-I/O chain. The answer may still omit details or generate unsupported claims; verify the response against the original text.

PROGRAM 8: SENTIMENT ANALYSIS USING A HUGGING FACE PIPELINE

Aim: To apply a pretrained Hugging Face sentiment-analysis pipeline to sample customer-feedback sentences and classify them as positive or negative.

Theory: Sentiment analysis is a text-classification task that estimates the emotional polarity of a sentence or review. Hugging Face pipelines hide much of the model-tokenisation and inference code behind a compact interface. The returned score is a model confidence score, not a calibrated guarantee of correctness.

Algorithm / Procedure

- Select a real-world application such as e-commerce customer review monitoring.
- Load a pretrained sentiment-analysis pipeline.
- Pass a list of sample feedback sentences to the pipeline.
- Print the sentiment label and score for each sentence.
- Review ambiguous or mixed-sentiment sentences manually.

Tools and Libraries: Python 3.10+, transformers, and PyTorch. Internet access is required on the first model download.

Program / Practical Procedure

```
from transformers import pipeline
classifier = pipeline(
    task="sentiment-analysis",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english"
)
reviews = [
    "The battery lasts all day and the laptop is excellent.",
    "The parcel arrived late and the product was damaged.",
    "Setup was easy, but the mobile app crashes often."
]
results = classifier(reviews)
for review, result in zip(reviews, results):
    print(f"Review: {review}")
    print(f"Sentiment: {result['label']} | Score: {result['score']:.3f}\n")
```

Expected Output / Observations

The model returns a sentiment label and confidence score for each review. The first sentence is likely positive, the second likely negative, and the mixed third sentence may be simplified into one label by the model.

Result / Conclusion

The experiment demonstrates pretrained text classification in a practical feedback-monitoring scenario. Mixed sentiment, sarcasm, and domain-specific phrasing can produce errors, so human review remains useful for important decisions.

PROGRAM 9: STRUCTURED INSTITUTION INFORMATION EXTRACTION USING LANGCHAIN AND PYDANTIC

Aim: To accept an institution name, retrieve its Wikipedia text, and use a LangChain application with a Pydantic schema and custom output parser to return founder, year founded, branches, employee count, and a concise four-line summary.

Theory: The Wikipedia API supplies source text, while a language model extracts the requested fields. A Pydantic model defines the expected data types and field names. A custom LangChain output parser converts the model response from JSON into the Pydantic model, rejecting malformed structures. Values missing from the source should be explicitly reported as unavailable instead of guessed.

Algorithm / Procedure

- Accept an institution name from the user.
- Use the MediaWiki API to retrieve the article extract.
- Define the output schema in Pydantic.
- Create a prompt that requests only JSON and instructs the model not to invent missing facts.
- Pipe the prompt to a chat model and then to a custom output parser.
- Display the validated object and its source URL.

Tools and Libraries: Python 3.10+, requests, pydantic, langchain-core, and langchain-openai. Set `OPENAI_API_KEY` before execution.

Program / Practical Procedure

```
import json, os, re, requests
from pydantic import BaseModel, Field
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import BaseOutputParser
from langchain_openai import ChatOpenAI
class InstitutionInfo(BaseModel):
    institution: str
    founder: str
    founded_year: str
    current_branches: list[str] = Field(default_factory=list)
    employee_count: str
    summary: str  # concise four-line summary
    source_url: str
class InstitutionParser(BaseOutputParser):
    """Parse JSON text and validate it with Pydantic."""
    def parse(self, text: str) -> InstitutionInfo:
        cleaned = re.sub(r"^```(?:json)?\s*|\s*```$", "", text.strip(), flags=re.I | re.M)
        data = json.loads(cleaned)
        return InstitutionInfo.model_validate(data)
    @property
    def _type(self) -> str:
        return "institution_pydantic_json"
name = input("Institution name: ").strip()
api_url = "https://en.wikipedia.org/w/api.php"
params = {"action": "query", "prop": "extracts", "explaintext": 1,
          "titles": name, "format": "json", "redirects": 1}
response = requests.get(api_url, params=params, timeout=20,
                        headers={"User-Agent": "CSL75-Lab/1.0"})
response.raise_for_status()
pages = response.json()["query"]["pages"]
page = next(iter(pages.values()))
article = page.get("extract", "")
if not article:
    raise ValueError("No Wikipedia article text was found.")
canonical = page.get("title", name)
source_url = "https://en.wikipedia.org/wiki/" + canonical.replace(" ", "_")
prompt = PromptTemplate.from_template(
    "Extract facts about the institution from the article. Do not guess. "
    "For missing details write 'Not reported in source'. Return only valid JSON "
    "with keys: institution, founder, founded_year, current_branches (array), "
    "employee_count, summary (four concise lines), source_url.\n"
    "Institution: {name}\nSource URL: {url}\nArticle:\n{article}"
)
chain = prompt | ChatOpenAI(model="gpt-4.1-mini", temperature=0) | InstitutionParser()
result = chain.invoke({"name": canonical, "url": source_url, "article": article[:14000]})
print(result.model_dump_json(indent=2))
```

Expected Output / Observations

The application prints a validated JSON object containing the requested fields and source URL. Wikipedia may not report the number of employees or a complete list of current branches; these should remain marked as unavailable when the source does not provide them.

Illustrative JSON Structure

```
{
  "institution": "<retrieved Wikipedia title>",
  "founder": "Not reported in source",
  "founded_year": "Not reported in source",
  "current_branches": [],
  "employee_count": "Not reported in source",
  "summary": "Four concise lines grounded in article text.",
  "source_url": "<Wikipedia article URL>"
}
```

Result / Conclusion

The experiment combines public-source retrieval, schema-guided prompting, and structured parsing. Because both Wikipedia content and model extraction can contain omissions, verify important fields against official institutional sources.

Validation Checks

Field	Validation point
Founder / year	Each value must be supported by the retrieved article; otherwise mark it unavailable.
Branches	Return a list and check whether entries refer to actual campuses or branches.
Employee count	Do not infer a number from unrelated statistics or an undated statement.
Summary / URL	Keep the summary concise and confirm that the URL opens the correct article.

PROGRAM 10: RETRIEVAL-BASED IPC QUESTION-ANSWERING CHATBOT

Aim: To load an Indian Penal Code document, split it into chunks, create an embedding-based searchable index, retrieve relevant passages, and answer user questions through a question-answering pipeline.

Theory: Retrieval-augmented question answering first searches source documents and then uses relevant passages to answer a query. This implementation creates dense sentence embeddings for document chunks and stores them in an in-memory matrix. Cosine similarity retrieves semantically close passages. A pretrained extractive QA model then identifies an answer within the highest-ranked context. Source passages are displayed to make the answer easier to verify.

Algorithm / Procedure

- Obtain an IPC document in text or searchable PDF form.
- Extract and clean the text; scanned PDFs may require OCR before this step.
- Split the text into overlapping chunks to retain context at boundaries.
- Encode chunks using a pretrained sentence-transformer embedding model.
- Embed each user question and retrieve the most similar chunks by cosine similarity.
- Use an extractive QA model on the retrieved passages and print the answer plus the best source chunk.
- Exit the interactive loop when the user enters `exit`.

Tools and Libraries: Python 3.10+, sentence-transformers, transformers, torch, numpy, and pypdf for PDF files. First use downloads model weights.

Program / Practical Procedure

```
import argparse, re
from pathlib import Path
import numpy as np
from sentence_transformers import SentenceTransformer
from transformers import pipeline
def load_text(path):
    path = Path(path)
    if path.suffix.lower() == ".pdf":
        from pypdf import PdfReader
        text = "\n".join(p.extract_text() or "" for p in PdfReader(path).pages)
    else:
        text = path.read_text(encoding="utf-8", errors="replace")
    text = re.sub(r"\s+", " ", text).strip()
    if not text:
        raise ValueError("No text extracted; OCR may be required for a scanned PDF.")
    return text
def chunk_text(text, size=900, overlap=150):
    return [text[i:i + size] for i in range(0, len(text), size - overlap)]
parser = argparse.ArgumentParser()
parser.add_argument("--document", required=True, help="Path to IPC .txt or .pdf")
args = parser.parse_args()
chunks = chunk_text(load_text(args.document))
print("Chunks indexed:", len(chunks))
embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
embeddings = embedder.encode(chunks, normalize_embeddings=True)
qa = pipeline("question-answering", model="distilbert/distilbert-base-cased-distilled-squad")
print("Ask questions about the loaded document (type exit to stop).")
while True:
    question = input("Question: ").strip()
    if question.lower() in {"exit", "quit"}:
        break
    if not question:
        continue
    qvec = embedder.encode([question], normalize_embeddings=True)[0]
    scores = embeddings @ qvec  # cosine similarity due to normalised vectors
    best = np.argsort(scores)[::-1][:3]
    candidates = []
    for idx in best:
        answer = qa(question=question, context=chunks[int(idx)])
        candidates.append((answer["score"], answer["answer"], int(idx)))
    score, answer, idx = max(candidates)
    print("Answer:", answer)
    print(f"Retrieved chunk {idx} (similarity {scores[idx]:.3f})")
    print("Source passage:", chunks[idx][:700], "\n")
```

Expected Output / Observations

After indexing, the chatbot accepts questions and prints an extractive answer, a similarity score, and a source-passage preview. Output quality depends on the content and text extraction quality of the supplied IPC document.

Suggested Test Queries and Checks

Test query type	What to verify
Known section number	The retrieved passage should include the requested section or heading.
Paraphrased question	A semantically related passage should be retrieved even when wording differs.
Unrelated question	The system should not be trusted when no relevant passage is returned.
Source validation	Compare the answer with the displayed source passage and the official document.

Result / Conclusion

The program demonstrates embedding-based retrieval with a QA model. It is an educational prototype, not a legal-advice system. The Indian Penal Code is a historical statute replaced for most current criminal-law purposes by newer legislation from 2024; verify applicable law with official, current sources.

Failure Modes and Mitigations

Possible issue	Mitigation
Scanned PDF	Run OCR first or use a searchable text/PDF file.
Weak retrieval	Try the exact section number, a shorter query, or alternate wording.
Unsupported answer	Inspect the returned source passage before accepting the answer.
Outdated law	Consult official current legal sources; this lab prototype is not legal advice.

PROGRAM 11: TIME-SERIES FORECASTING USING A PYTORCH RNN

Aim: To generate a noisy sine-wave series, form fixed-length input-output sequences, train an `nn.RNN` followed by an `nn.Linear` prediction layer using MSE and Adam, and plot forecasts.

Theory: A recurrent neural network processes a sequence while maintaining a hidden state that carries information from earlier time steps. A sliding window of past observations is used to predict the following point. Mean Squared Error measures the difference between the prediction and target, and Adam updates the weights. The final portion of the time series is held out for evaluation.

Algorithm / Procedure

- Generate a sine wave and add Gaussian noise; set a random seed for reproducibility.
- Create sequence windows of a fixed length and use the next value as the target.
- Split training and evaluation windows chronologically.
- Define an RNN and a fully connected output layer.
- Train with MSE loss and Adam; print loss at intervals.
- Predict the series, calculate held-out MSE, and save a plot comparing predictions with the original series.

Tools and Libraries: Python 3.10+, torch, numpy, matplotlib.

Program / Practical Procedure

```
import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
torch.manual_seed(42)
np.random.seed(42)
device = torch.device("mps" if torch.backends.mps.is_available() else
                      "cuda" if torch.cuda.is_available() else "cpu")
time = np.linspace(0, 60, 700, dtype=np.float32)
values = np.sin(time) + np.random.normal(0, 0.12, len(time)).astype(np.float32)
seq_len = 20
def sequences(series):
    x, y = [], []
    for i in range(len(series) - seq_len):
        x.append(series[i:i + seq_len])
        y.append(series[i + seq_len])
    return (torch.tensor(np.array(x)).unsqueeze(-1),
            torch.tensor(np.array(y)).unsqueeze(-1))
split = int(0.8 * len(values))
x_train, y_train = sequences(values[:split])
x_all, y_all = sequences(values)
loader = DataLoader(TensorDataset(x_train, y_train), batch_size=32, shuffle=True)
class RNNForecast(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(1, 32, batch_first=True)
        self.fc = nn.Linear(32, 1)
    def forward(self, x):
        out, _ = self.rnn(x)
        return self.fc(out[:, -1, :])
model = RNNForecast().to(device)
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
for epoch in range(1, 61):
    total = 0.0
    model.train()
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(xb)
    if epoch == 1 or epoch % 10 == 0:
        print(f"Epoch {epoch:02d}, MSE: {total / len(loader.dataset):.5f}")
model.eval()
with torch.no_grad():
    pred = model(x_all.to(device)).cpu().numpy().ravel()
actual = y_all.numpy().ravel()
test_start = max(0, split - seq_len)
```

```
print("Held-out test MSE:", float(np.mean((pred[test_start:] - actual[test_start:]) ** 2)))
plt.figure(figsize=(10, 5))
plt.plot(time, values, label="Noisy sine series")
plt.plot(time[seq_len:], pred, label="RNN prediction")
plt.axvline(time[split], linestyle="--", label="Train/test boundary")
plt.legend(); plt.xlabel("Time"); plt.ylabel("Value")
plt.title("RNN Time-Series Forecasting")
plt.tight_layout(); plt.savefig("rnn_forecast.png", dpi=160); plt.close()
print("Saved plot: rnn_forecast.png")
```

Expected Output / Observations

The terminal prints training MSE at the first and each tenth epoch, a held-out test MSE, and a message that `rnn_forecast.png` was saved. The plot contains the noisy series, RNN prediction, and train/test boundary.

Evaluation Checklist

Metric / visual check	Interpretation
Training MSE	It should generally trend downward; a flat or rising curve can indicate poor settings.
Held-out MSE	Record the numerical value printed by the program and compare runs consistently.
Forecast plot	Inspect whether peaks, troughs, and overall trend are followed.
Train/test split	Ensure final observations are held out and not used to fit the model.

Result / Conclusion

The RNN completes the requested time-series forecasting pipeline. Since the data are synthetic, the experiment verifies the model workflow rather than proving performance on a real forecasting task.

PROGRAM 12: CHARACTER-LEVEL TEXT GENERATION USING AN LSTM

Aim: To prepare text as character sequences, train an LSTM to predict the next character, and generate new text from a seed phrase while experimenting with sequence length, hidden units, and epochs.

Theory: Long Short-Term Memory networks are recurrent neural networks designed to retain useful information over longer sequences using gates. Character-level generation maps each character to an integer ID, learns next-character patterns, and samples repeatedly from the model to extend a seed phrase. Temperature controls sampling randomness. A small training corpus demonstrates the method but is unlikely to produce coherent long-form writing.

Algorithm / Procedure

- Load a text corpus from a file or use the included demonstration passage.
- Build character-to-index and index-to-character mappings.
- Create overlapping input sequences and targets shifted by one character.
- Define an embedding layer, LSTM layer, and linear output layer.
- Train using cross-entropy loss and Adam, printing loss every few epochs.
- Start from a seed phrase and sample one character at a time to generate new text.
- Adjust sequence length, hidden size, epochs, and temperature to observe their effects.

Tools and Libraries: Python 3.10+ and torch. A larger, appropriately licensed text file may be supplied for improved training.

Program / Practical Procedure

```
import argparse, random
from pathlib import Path
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader
DEMO = ("The morning was quiet. A curious student opened a book and began to learn. "
        "Every question led to another idea, and each small discovery encouraged "
        "the student to continue exploring the world of language and stories. " * 40)
class CharDataset(Dataset):
    def __init__(self, ids, length):
        self.ids = torch.tensor(ids, dtype=torch.long)
        self.length = length
    def __len__(self):
        return len(self.ids) - self.length
    def __getitem__(self, i):
        return self.ids[i:i+self.length], self.ids[i+1:i+self.length+1]
class CharLSTM(nn.Module):
    def __init__(self, vocab, hidden=96):
        super().__init__()
        self.embed = nn.Embedding(vocab, 32)
        self.lstm = nn.LSTM(32, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, vocab)
    def forward(self, x):
        out, _ = self.lstm(self.embed(x))
        return self.fc(out)
parser = argparse.ArgumentParser()
parser.add_argument("--text-file", type=Path)
parser.add_argument("--epochs", type=int, default=15)
parser.add_argument("--seq-len", type=int, default=60)
parser.add_argument("--hidden", type=int, default=96)
parser.add_argument("--seed-text", default="The morning")
args = parser.parse_args()
random.seed(42); torch.manual_seed(42)
text = args.text_file.read_text(encoding="utf-8") if args.text_file else DEMO
chars = sorted(set(text))
to_id = {ch: i for i, ch in enumerate(chars)}
from_id = {i: ch for ch, i in to_id.items()}
ids = [to_id[ch] for ch in text]
dataset = CharDataset(ids, args.seq_len)
loader = DataLoader(dataset, batch_size=64, shuffle=True)
device = torch.device("mps" if torch.backends.mps.is_available() else
                      "cuda" if torch.cuda.is_available() else "cpu")
model = CharLSTM(len(chars), args.hidden).to(device)
opt = torch.optim.Adam(model.parameters(), lr=0.003)
loss_fn = nn.CrossEntropyLoss()
for epoch in range(1, args.epochs + 1):
    total = 0.0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
```

```
        opt.zero_grad(); logits = model(x)
        loss = loss_fn(logits.reshape(-1, len(chars)), y.reshape(-1))
        loss.backward(); torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step(); total += loss.item() * len(x)
    if epoch == 1 or epoch % 5 == 0 or epoch == args.epochs:
        print(f"Epoch {epoch:02d}, cross-entropy: {total / len(dataset):.4f}")
seed = "".join(ch if ch in to_id else " " for ch in args.seed_text)
context = torch.tensor([[to_id.get(ch, to_id[chars[0]]) for ch in seed]], device=device)
model.eval(); generated = seed
with torch.no_grad():
    for _ in range(300):
        logits = model(context[:, -args.seq_len:])[:, -1, :] / 0.8
        probs = torch.softmax(logits, dim=-1)
        next_id = int(torch.multinomial(probs, 1).item())
        generated += from_id[next_id]
        context = torch.cat([context, torch.tensor([[next_id]], device=device)], dim=1)
print("\nGenerated text:\n", generated)
```

Expected Output / Observations

The program prints the corpus vocabulary implicitly through training data preparation, the cross-entropy loss at selected epochs, and generated text beginning with the seed phrase. Generation quality will be limited by the demonstration corpus.

Hyperparameter Observation Guide

Parameter	Effect to observe
Sequence length	Longer context may capture more structure but increases input size.
Hidden units	More units increase capacity and training cost; they do not guarantee better text.
Epochs	More training may reduce loss but can overfit a small corpus.
Temperature	Lower values tend to be more predictable; higher values usually increase variety.

Result / Conclusion

The exercise demonstrates next-character prediction with an LSTM and autoregressive generation. Larger and more varied text data plus tuning are required for more coherent output; generated text may be repetitive or inaccurate.

OVERALL CONCLUSION

The twelve exercises cover multimodal generation, prompt enrichment, neural-network classification, summarisation, pretrained and custom word embeddings, LangChain prompt/model workflows, sentiment analysis, structured information extraction, retrieval-based question answering, RNN forecasting, and LSTM text generation. They illustrate different stages of an AI workflow: selecting data and prompts, representing information, invoking models, evaluating outputs, and documenting limitations. Outputs should be reviewed critically and verified against the original source when accuracy matters.

REFERENCES

- CSL75: Skill Enhancement Laboratory - Generative AI, laboratory exercises 1-12, course sheet provided for this assignment.
- Gensim documentation: https://radimrehurek.com/gensim/
- TensorFlow Keras documentation: https://www.tensorflow.org/guide/keras
- Hugging Face Transformers documentation: https://huggingface.co/docs/transformers/
- LangChain documentation: https://python.langchain.com/docs/
- Cohere API documentation: https://docs.cohere.com/
- Pydantic documentation: https://docs.pydantic.dev/
- Sentence Transformers documentation: https://www.sbert.net/
- PyTorch documentation: https://pytorch.org/docs/stable/index.html
- MediaWiki API documentation: https://www.mediawiki.org/wiki/API:Main_page
- Ministry of Home Affairs, Government of India, New Criminal Laws portal (Bharatiya Nyaya Sanhita, 2023): https://www.mha.gov.in/en/commoncontent/new-criminal-laws

REPRODUCIBILITY AND SUBMISSION NOTES

- Install each program’s dependencies in a virtual environment and retain the version output when required.
- For Programs 1 and 2, record the actual generative-AI tool, prompt, generated artefacts, and comparative observations.
- For Program 7, use a publicly accessible text file in Google Drive and store the Cohere API key in an environment variable.
- For Program 9, store the language-model API key securely and verify the extracted institutional fields against official sources.
- For Program 10, provide a searchable IPC text/PDF file. The prototype is for education and is not legal advice; check current law using official sources.
- Program output snippets are illustrative; add actual run screenshots where your instructor requires execution evidence.